

Week2 > LinearSearch.java > LinearSearch > main(String[])

```
1  package Week2;
2  import java.util.*;
3
4  public class LinearSearch{
    Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          int N = sc.nextInt();
8          int[] transactions = new int[N];
9
10         for(int i=0;i<N;i++){
11             transactions[i] = sc.nextInt();
12         }
13
14         int suspiciousId = sc.nextInt();
15
16         boolean found = false;
17
18         for(int i=0;i<N;i++){
19             if(transactions[i] == suspiciousId){
20                 found = true;
21                 break;
22             }
23         }
24
25         if(found){
26             System.out.println(x: "Fraud Transaction found");
27         }else{
28             System.out.println(x: "Fraud Transaction NOT found");
29         }
30
31         sc.close();
32     }
33 }
```



Search



```

1  package Week2;
2  import java.util.*;
3
4  public class BinarySearch {
    Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          int N = sc.nextInt();
8          int[] logID = new int[N];
9
10         for(int i=0;i<N;i++){
11             logID[i] = sc.nextInt();
12         }
13
14         int targetID = sc.nextInt();
15         int low = 0;
16         int high = logID.length-1;
17         boolean found = false;
18
19         while(low<high){
20             int mid = low+(high-low)/2;
21             if(logID[mid]==targetID){
22                 found= true;
23                 break;
24             }else if(logID[mid]<targetID){
25                 low=mid+1;
26             }else{
27                 high=mid-1;
28             }
29         }
30
31         if(found){
32             System.out.println(x: "log found");
33         }else{
34             System.out.println(x: "log not found");
35         }
36
37         sc.close();
38     }
39 }
40

```


Week2 > BubbleSort.java > BubbleSort > main(String[])

```
1  package Week2;
2  import java.util.*;
3
4  public class BubbleSort {
    Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          System.out.println(x: "Enter number of salary:");
8          int N = sc.nextInt();
9
10         int[] salary = new int[N];
11         System.out.println(x: "Enter salary:");
12         for(int i=0; i<N; i++){
13             salary[i] = sc.nextInt();
14         }
15         // sorting using bubble sort
16         boolean swapped = false;
17         for(int i=0; i<N; i++){
18             for(int j=0; j<N-1-i; j++){
19                 if(salary[j]>salary[j+1]){
20                     int temp = salary[j+1];
21                     salary[j+1]=salary[j];
22                     salary[j]=temp;
23                     swapped=true;
24                 }
25             }
26             if(!swapped){
27                 break;
28             }
29         }
30         //printing of sorted array
31         System.out.println(x: "sorted salary: ");
32         for(int i=0; i<N; i++){
33             System.out.println(salary[i]);
34         }
35         sc.close();
36     }
37 }
38
```

Week2 > J SelectionSort.java > SelectionSort > main(String[])

```
1  package Week2;
2  import java.util.*;
3
4  public class SelectionSort {
    Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          System.out.println(x: "Enter number of priority:");
8          int N = sc.nextInt();
9
10         int[] priority = new int[N];
11         System.out.println(x: "Enter priority:");
12         for(int i=0;i<N;i++){
13             priority[i] = sc.nextInt();
14         }
15
16         // sorting using selection
17         for(int i=0;i<priority.length-1;i++){
18             int minPos = i;
19             for(int j=i+1;j<priority.length;j++){
20                 if(priority[minPos]>priority[j]){
21                     minPos=j;
22                 }
23             }
24             int temp =priority[minPos];
25             priority[minPos]=priority[i];
26             priority[i]=temp;
27         }
28
29         //printing of sorted array
30         System.out.println(x: "sorted priority: ");
31         for(int i=0;i<N;i++){
32             System.out.println(priority[i]);
33         }
34         sc.close();
35     }
36 }
37
38
```


Week2 > J InsertionSort.java > ...

```
1  package Week2;
2  import java.util.*;
3
4  public class InsertionSort {
    Run | Debug
5      public static void main(String[] args) {
6          Scanner sc = new Scanner(System.in);
7          System.out.println(x: "Enter number of response time:");
8          int N = sc.nextInt();
9
10         int[] response = new int[N];
11         System.out.println(x: "Enter response time:");
12         for(int i=0;i<N;i++){
13             response[i] = sc.nextInt();
14         }
15
16         // sorting using innsertion sort
17         for(int i=1;i<response.length;i++){
18             int curr = response[i];
19             int prev= i-1;
20             while(prev>=0 && response[prev]>curr){
21                 response[prev+1]=response[prev];
22                 prev--;
23             }
24             response[prev+1]=curr;
25         }
26
27
28         //printing of sorted array
29         System.out.println(x: "sorted response time: ");
30         for(int i=0;i<N;i++){
31             System.out.println(response[i]);
32         }
33         sc.close();
34     }
35 }
36
37
38
```

Integer.MAX_VALUE (10)

update(marks, a);
System.out.println(a);

for (int i=0; i < marks.length; i++) {
 System.out.println(marks[i]);
}

Linear Search

public static int linearSearch (int numbers[], int key) {

for (int i=0; i < numbers.length; i++) {

if (numbers[i] == key) {

return i;

}

return -1;

}

public static void main (String[] args) {

int numbers[] = {2, 4, 6, 8, 10, 12, 14, 16};

int key = 10;

int index = linearSearch (numbers, key);

if (index == -1) {

System.out.println("Not found");

} else {

System.out.println("Key is at index: " + index);

}

Binary Search

```
public static int binarySearch(int numbers[], int key) {  
    int start = 0, end = numbers.length - 1;  
  
    while (start <= end) {  
        int mid = (start + end) / 2;  
  
        if (numbers[mid] == key) {  
            return mid;  
        }  
        if (numbers[mid] < key) {  
            start = mid + 1;  
        }  
        else {  
            end = mid - 1;  
        }  
    }  
    return -1;  
}
```

```
public static void main (String args[]) {  
    int numbers[] = { 2, 4, 6, 8, 10, 12, 14 };  
    int key = 4;  
    System.out.println("index for key is: " + binarySearch(numbers, key));  
}
```

Bubble Sort

```
public static void bubbleSort(int arr[]) {  
    int swapped = 0;  
    for (int i = 0; i < arr.length - 1; i++) {  
        for (int j = 0; j < arr.length - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
                swapped++;  
            }  
            if (swapped == 0) {  
                break;  
            }  
        }  
    }  
}
```

$O(n)$
 $A = O(n^2)$
 $\omega = O(1)$

```
public static void printArr(int arr[]) {  
    for (int i = 0; i < arr.length; i++) {  
        System.out.print(arr[i] + " ");  
    }  
    System.out.println();  
}
```

```
public static void main(String args[]) {  
    int arr[] = {5, 4, 1, 3, 2};  
    bubbleSort(arr);  
    printArr(arr);  
}
```


Selection Sort

↳ pick the smallest (from unsorted), put it at the beginning.

```
public static void selectionSort(int arr[]) {  
    for (int i = 0; i < arr.length - 1; i++) {  
        int minPos = i;  
        for (int j = i + 1; j < arr.length; j++) {  
            if (arr[minPos] > arr[j]) {  
                minPos = j;  
            }  
        }  
        int temp = arr[minPos];  
        arr[minPos] = arr[i];  
        arr[i] = temp;  
    }  
}
```

$O(n^2)$
B =
A =
W =

```
public static void main(String args[]) {  
    int arr[] = {5, 4, 1, 2, 3};  
    selectionSort(arr);  
    printArr(arr);  
}
```

unsorted part / 4 places

```
selectionSort(arr);  
printArr(arr);
```

```
}
```

Insertion Sort - pick an element (from unsorted part) & place in the right pos in sorted part.

```
public static void insertionSort(int arr[]) {  
    for (int i = 1; i < arr.length; i++) {  
        int curr = arr[i];  
        int prev = i - 1;  
        while (prev >= 0 & arr[prev] > curr) {  
            arr[prev + 1] = arr[prev];  
            prev --;  
        }  
        arr[prev + 1] = curr;  
    }  
}
```

```
}
```

```
}
```

```
main {  
    insertionSort(arr);  
    printArr(arr);  
}
```

```
}
```